

Série d'exercices D

Fonctions d'ordre supérieur

1. Ecrivez une fonction qui prend un vecteur et une liste d'entiers en paramètre, et qui retourne un `Vector[List[Int]]`, où chaque liste du vecteur est la liste initiale multipliée par une des composantes du vecteur.

2. Utilisez la fonction `reduceLeft` pour calculer la valeur maximum d'une liste d'entiers.

3. Évaluez `1.to(10).reduceLeft(_ * _)`.

Qu'est-ce que vous obtenez ? Écrire une fonction qui calcule $n!$ de cette façon.

4. Maintenant, on aimerait calculer 2^n avec la même astuce. Comment obtenir une séquence de n copie d'un nombre ? Astuce : `map`.

Quelle est votre fonction qui calcule 2^n ?

5. Étant donné une `Seq[String]`, comment peut-on utiliser `reduceLeft` pour les concaténer avec des espaces entre les deux ? Écrivez une fonction `catSpace` qui fait cela.

Par exemple, `catSpace(Vector("I", "have", "a", "dream"))` devrait retourner la chaîne "I have a dream".

6. En utilisant `foldLeft` (ou `foldRight`) écrivez une fonction `reverse` pour inverser l'ordre des éléments d'une liste.

7. Étant donné une matrice représentée comme une liste de listes, écrivez une fonction qui renvoie la première, ou une colonne arbitraire de la matrice.

```
def firstColumn(xs: List[List[Int]]): List[Int] = ???  
def column(xs: List[List[Int]], col: Int): List[Int] = ???
```

8. Étant donné une matrice représentée comme une liste de listes, écrivez une fonction qui renvoie la diagonale de la matrice.

```
def diagonal(xs: List[List[Int]]): List[Int] = ???
```

9. Étant donné une matrice représentée comme une liste de listes, écrivez une fonction qui vérifie si la matrice a une ligne composée uniquement de zéros :

```
def hasZeroRow(matrix: List[List[Int]]): Boolean = ???
```

10. Ecrivez un test pour savoir si un nombre est premier. Un nombre n est premier si les seuls diviseurs de n sont 1 et n lui-même. Restez simple, l'efficacité n'est pas importante pour l'instant :

```
def isPrime(x: Int): Boolean = ???
```

11. Étant donné la liste des chaînes, recherchez toutes les lignes plus longues qu'une longueur minimale

```
def linesLonger(lines: List[String], len: Int): List[String] = ???
```

12. Étant donné la liste des chaînes, trouvez la longueur de la ligne la plus longue. Écrivez cette fonction à l'aide de l'opérateur `foldLeft`. Pouvez-vous également utiliser l'opérateur `reduceLeft` ?

```
def longestLineLength(lines: List[String]): Int = ???
```

13. Étant donné la liste des chaînes, supprimez toutes les lignes vides.

```
def elimEmptyLines(lines: List[String]): List[String] = ???
```

14. Étant donné la liste des chaînes, recherchez la ligne la plus longue (s'il y en a plusieurs de même taille retourner la dernière).

```
def longestLine(lines: List[String]): String = ???
```

15. En utilisant la fonction `foldRight`, implémenter la fonction `compress` qui permet d'éliminer les éléments successifs identiques dans une liste et de retourner une nouvelle liste sans les répétition des éléments consécutifs (voir l'exemple fourni).

```
compress(List('a', 'a', 'a', 'a', 'b', 'c', 'c', 'a', 'a', 'd', 'e', 'e', 'e', 'e'))  
//> res0: List[Char] = List('a', 'b', 'c', 'a', 'd', 'e')
```

```
def compress[A](ls: List[A]): List[A] = ???
```

16. En utilisant la fonction `foldLeft`, implémenter la méthode `averageOfDoubles` qui calcule la moyenne d'une liste non-vide de nombres réels. Attention : Vous ne devez pas utiliser d'autres fonctions que `foldLeft` et la division.

Exemple d'application :

```
averageOfDoubles(List(2.0, 2.5, 4.5))  
//res5: Double = 3.0
```

```
def averageOfDoubles(l: List[Double]): Double = ???
```